How the Chapter 4 Data Was Created and Recorded

A complete, reproducible account of the measurement run, the analysis toolchain, and every figure and table it produced

Item	Value
Robot	7-DOF arm (joint_1 … joint_7), tip link "ee", base "base_link"
Backend	Gazebo Ignition + ros2_control (ign_ros2_control), simulation only
Drawn shape	Single closed curved stroke, ≈ 17 mm across ("circle" run)
Source recording	rosbag2 "draw_run" (sqlite3), 10,002 messages, 101.1 s
Artefacts produced	Figures 4.1, 4.3, 4.4, 4.5 (PNG + PDF); Tables 4.1, 4.2 (DOCX)
Analysis location	src/arm_bot/analysis/ (run with python3, /opt/ros only)

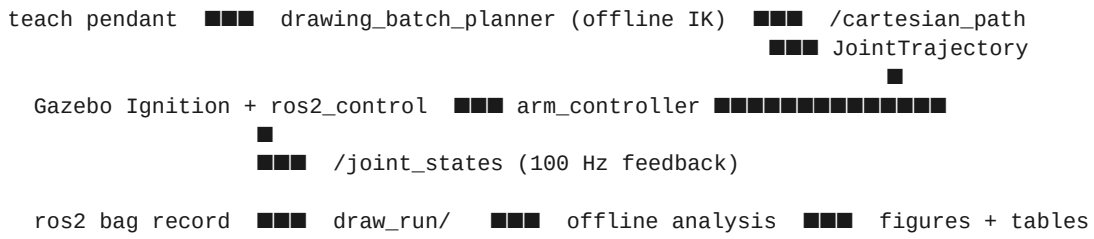
This document explains the methodology only — what was run, in what order, with which parameters, and how each number was obtained — so the run can be reproduced and defended.

1 Overview and design principle

All Chapter 4 quantitative results come from a **single recording of one drawing run**. The teach pendant commanded the arm to draw a small closed stroke; while it executed, a ROS 2 bag captured the commanded path, the measured joint feedback, and the robot model. Every figure and the timing table are then derived *offline* from that one bag, so they are mutually consistent (same motion, same time base) and the run can be replayed at any time without the simulator.

The analysis is deliberately decoupled from the live system: the scripts in `src/arm_bot/analysis/` read the URDF straight out of the bag and rebuild the forward-kinematics chain themselves, so they need only a stock ROS install on the path (`/opt/ros`) — not the built workspace overlay. They are plain `python3` scripts, not `ros2 run` nodes.

Pipeline at a glance



2 The recording (data capture)

2.1 Bringing up the system

The simulation backend was started through the pendant's composed launch graph (Gazebo, ros2_control, RViz, the FK/IK nodes and the batch drawing planner):

```
ros2 launch arm_bot pendant_backend.launch.py
```

The terminals that record and analyse must run under **bash** (the ROS `setup.bash` files are bash; the user's default shell is fish). The pendant application itself re-execs under bash automatically.

2.2 Recording command

A rosbag2 recording was started **before** clicking *Send* in the pendant Drawing tab. This ordering is critical: `/cartesian_path` is published **once** (a one-shot latched message that carries the whole resampled path), so a recorder started after Send would miss it.

```
ros2 bag record -o draw_run \
    /joint_states /cartesian_path /robot_description
```

The drawing was then sent and allowed to run to completion — move-to-begin pose, dwell, draw the stroke, lift — before stopping the recorder with Ctrl-C.

2.3 What the bag contains

The resulting bag (`draw_run/draw_run_0.db3`, sqlite3 storage) holds exactly three topics:

Topic	Type	Msgs	Role in the analysis
/joint_states	sensor_msgs/JointState	10,000	Measured (executed) joint feedback at ~100 Hz — drives Figs 4.3-4
/cartesian_path	geometry_msgs/PoseArray	1	The commanded drawing path (resampled waypoints, already in pap
/robot_description	std_msgs/String	1	The URDF — used to rebuild the FK chain and to read joint_6 limits

Total: 10,002 messages over a duration of 101.1 s. The 10,000 `/joint_states` samples at ~100 Hz confirm the controller dispatch / feedback rate quoted in Table 4.1.

2.4 Coordinate convention captured in the bag

The `/cartesian_path` poses are published by the batch planner **already in paper-plane coordinates** (the position x,y,z are the raw waypoint metres on the drawing plane), *not* base-frame end-effector targets. Only the executed path needs transforming from the base frame onto the paper plane. Getting this right was a fix during the first real run — see §6.

3 Analysis toolchain

Four scripts live in `src/arm_bot/analysis/`. All run with plain `python3` after only `source /opt/ros/humble/setup.bash` (no workspace overlay needed — the FK chain is self-contained).

Script	Produces	Input
plot_commanded_vs_executed.py	Figure 4.1 (standalone)	draw_run bag
make_thesis_figures.py	Figures 4.1, 4.3, 4.4, 4.5	draw_run bag
make_table_4_1.py	Table 4.1 (timing) DOCX	measured values
make_table_4_2.py	Table 4.2 (objectives) DOCX	authored / measured

`make_thesis_figures.py` imports the shared helpers (bag reader, URDF FK chain, paper-frame transform, tracking-error metric) from `plot_commanded_vs_executed.py` via importlib, so the four figures share one code path.

3.1 Commands actually run

```
# one source line, then everything is plain python3 (bash shell)
source /opt/ros/humble/setup.bash
cd ~/7dof_ws

# Figures 4.1, 4.3, 4.4, 4.5 — all four from the one bag
python3 src/arm_bot/analysis/make_thesis_figures.py draw_run

# Tables
python3 src/arm_bot/analysis/make_table_4_1.py \
        --out table_4_1_timing.docx
python3 src/arm_bot/analysis/make_table_4_2.py \
        --out table_4_2_objectives_vs_outcomes.docx
```

Each figure is written as both a 300 dpi PNG (for review) and a vector PDF (for LaTeX inclusion). Outputs land in the current directory unless `--outdir` is given.

4 How each figure was derived

4.1 Figure 4.1 — commanded vs executed path overlay

The headline accuracy figure. It overlays the path the pendant *commanded* against the path the arm *executed*, both expressed as pen-tip positions on the drawing plane in millimetres, and annotates the gap between them.

- **Commanded path**: the waypoints from the one-shot `/cartesian_path` PoseArray, scaled metres→mm (position × 1000). No transform — they are already paper coordinates.
- **Executed path**: for every recorded `/joint_states` sample, forward kinematics (the URDF chain, base→ee) gives the end-effector pose, which is mapped onto the paper plane by the exact transform the planner uses (pen offset along the EE-local pen axis, anchored at the pen tip at the begin-draw pose, rotated by the paper rotation).
- **Tracking error**: point-to-point distance between executed and commanded curves; the script prints and annotates RMS and max in mm.

- **Windowing**: the executed stream is trimmed to the drawing phase only — the move-to-begin (4 s), dwell (3 s), settle (0.5 s) and a 1 s approach are dropped using the `/cartesian_path` stamp as t=0, so the begin-pose swing is not counted as error.

Paper-frame parameters are baked in to match `pendant_backend.launch.py`: begin-draw joints [0, -0.7, 0, 1.4, 0.01, 0, 1], pen offset 100 mm, pen axis local [1,0,0], paper rotation 270°, no mirror. (These shift/rotate both curves identically, so a small mismatch only reorients the axes — it cannot corrupt the comparison.)

Result: clean overlay, tracking error **RMS 0.11 mm, max 0.61 mm** over ~2000 pen-down samples.

4.2 Figure 4.3 — joint position trajectories

Seven lines (joint_1 … joint_7) of measured joint angle vs time, read directly from `/joint_states`. The series is indexed **by joint name on every message**, because the bag's name order is not sorted (joint_3 / joint_4 appear swapped) — indexing by position would silently mislabel two joints. The time axis is zeroed at trajectory dispatch and the trailing static hold is auto-trimmed.

4.3 Figure 4.4 — joint velocity profiles

Seven lines of joint velocity vs time. The `velocity` field published in `/joint_states` is used directly (it was verified to match a numeric central-difference of the positions); a `--vel-source diff` fallback recomputes by differentiation if velocity is ever missing. **Measured peaks:** joint_4 0.61, joint_7 0.44, joint_2 0.30 rad/s.

4.4 Figure 4.5 — joint_6 vs its limits

joint_6 has the tightest travel on this arm, so it gets its own safety figure. The script reads joint_6's lower/upper limit straight from the URDF in the bag, shades the allowed band, draws the measured angle, and annotates the closest approach to a limit. **Result:** joint_6 used the range [-0.037, +0.020] rad against limits [-0.48, +0.26] rad — closest approach 0.24 rad, comfortably **within limits**.

Figure 4.2 (not produced from the bag) is a Gazebo screenshot of the traced pen path, captured separately with `ros2 launch arm_bot pendant_backend.launch.py enable_path_tracer:=true`.

5 How each table was built

5.1 Table 4.1 — timing and performance

Built as a DOCX by `make_table_4_1.py`. The values are measured from the drawing run two independent ways and cross-checked:

- **Per-waypoint IK solve time** — mean over the 45 path waypoints, obtained by replaying the planner's exact `solve_ik` on the recorded path offline, and confirmed live by a `[TIMING]` line the batch planner now prints. **0.65 ms** mean (1.27 ms max).
- **Batch generation time** — full path planning (spline resample + all IK): ≈ **30 ms**.
- **Dispatch rate** — the controller update rate, also visible as the 10 ms `/joint_states` period in the bag: **100 Hz**.

The offline replay also reported 45 waypoints, batch IK ~29 ms and a max IK residual of 7.6×10■■ — i.e. every waypoint converged.

5.2 Table 4.2 — objectives vs outcomes

This table is **authored, not measured** (no bag needed). `make_table_4_2.py` pairs each of the four Section 1.3 thesis objectives with the outcome that demonstrates it and the evidence (figure / table / chapter). The objective wording was taken verbatim from the thesis; the outcome cells embed the measured results from this run (notably the Fig 4.1 RMS 0.11 mm). The objectives frame the contribution as the **pendant as an operator / integration layer** — not the kinematics — so the drawing mode is presented as an end-to-end correctness benchmark, not as a novel IK algorithm.

6 Provenance, validation and fixes

The toolchain was exercised on the real recorded run, and two bugs were found and fixed in the process — both worth recording for defensibility:

- **Self-contained FK** — the FK chain class was inlined verbatim from the live FK node instead of imported from the `arm_bot` package, so the script runs with only `/opt/ros` sourced (the fish terminal had no workspace overlay, which had caused a ModuleNotFoundError).
- **Commanded-frame bug** — `/cartesian_path` is already in paper coordinates; an earlier version wrongly pushed it through the base→paper transform, so commanded and executed curves did not overlap and the error came out NaN. Fixed by scaling the commanded poses metres→mm only.

Note that the analysis touches two workspace files (a `[TIMING]` log line in the batch planner and a default-off `enable_path_tracer` launch arg) — both rebuilt cleanly and leave the pendant contract intact.

7 Reproducing the whole dataset from scratch

```
# 1. bring up the simulation backend
ros2 launch arm_bot pendant_backend.launch.py

# 2. in a bash terminal, START recording BEFORE clicking Send
ros2 bag record -o draw_run /joint_states /cartesian_path /robot_description
#    -> open the pendant, Drawing tab, draw a stroke, click Send,
#       let it finish (begin -> dwell -> draw -> lift), then Ctrl-C

# 3. generate every figure and table from that one bag
source /opt/ros/humble/setup.bash
python3 src/arm_bot/analysis/make_thesis_figures.py draw_run
python3 src/arm_bot/analysis/make_table_4_1.py --out table_4_1_timing.docx
python3 src/arm_bot/analysis/make_table_4_2.py --out table_4_2_objectives_vs_outcomes.docx
```

```
# (optional) Figure 4.2 path-tracer screenshot
ros2 launch arm_bot pendant_backend.launch.py enable_path_tracer:=true
```

Outputs: `figure_4_1.{png,pdf}` … `figure_4_5.{png,pdf}`, `table_4_1_timing.docx`, `table_4_2_objectives_vs_outcomes.docx`.

Summary of measured results

Quantity	Value	Source
End-effector tracking error	RMS 0.11 mm / max 0.61 mm	Fig 4.1
Per-waypoint IK solve time	0.65 ms mean (1.27 ms max)	Table 4.1
Batch trajectory generation	≈ 30 ms (45 waypoints)	Table 4.1
Controller dispatch rate	100 Hz	Table 4.1 / bag
Peak joint velocity	joint_4 0.61 rad/s	Fig 4.4
joint_6 range vs limits	[-0.037,+0.020] vs [-0.48,+0.26] rad	Fig 4.5
Max IK residual	7.6e-4 (all converged)	offline replay